Arbre de décision

Ludovic Huang, Augustin Grudet

Sommaire

1. Histoire des arbres de décisions

2. Construction d'arbre
2.1 Principe de construction
2.2 Iterative Dichotomiser 3 (ID3)
2.3 Classification and Regression Trees (CART)

3. Élagage d'arbre

4. Random Forest

5. Limites et conclusion

Contexte et objectif

- Problème : prédire une décision à partir d'observations.
- Idée : une suite de tests simples et interprétables.
- Avantages : explicable, rapide à entraîner, baseline solide.
- Objectif : comprendre ID3 et CART, critères de split, élagage.

Exemple introductif, Ross Quinlan 1989

But : construire un arbre de décision général en nous basant sur les observations ci-contre afin de savoir si on peut jouer au tennis.

Outlook	**Temperature**	**Humidity**	**Windy**	**Play**
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	Normal	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Table 1: Dataset introductif de Ross Quinlan (1989)

Sommaire

1. Histoire des arbres de décisions

2. Construction d'arbre
2.1 Principe de construction
2.2 Iterative Dichotomiser 3 (ID3)
2.3 Classification and Regression Trees (CART)

3. Élagage d'arbre

4. Random Forest

5. Limites et conclusion

Sommaire

1. Histoire des arbres de décisions

2. Construction d'arbre
2.1 Principe de construction
2.2 Iterative Dichotomiser 3 (ID3)
2.3 Classification and Regression Trees (CART)

3. Élagage d'arbre

4. Random Forest

5. Limites et conclusion

Principe de construction

- À chaque nœud, choisir un attribut pour séparer les classes.
- Le choix maximise un critère de pureté (entropie, Gini).
- Répéter jusqu'à un critère d'arrêt.

Critères de pureté

- Entropie : $H(S) = -\sum_i p_i \log_2(p_i)$
- Gain d'information : $IG(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$
- Gini : $G(S) = 1 - \sum_i p_i^2$

Attributs continus

- Pour un attribut numérique, tester des seuils t.
- On cherche le split binaire qui minimise Gini ou maximise le gain.
- CART gère nativement le numérique, ID3 nécessite une adaptation.

Sommaire

1. Histoire des arbres de décisions

2. Construction d'arbre
2.1 Principe de construction
2.2 Iterative Dichotomiser 3 (ID3)
2.3 Classification and Regression Trees (CART)

3. Élagage d'arbre

4. Random Forest

5. Limites et conclusion

Iterative Dichotomiser 3 (ID3)

Algorithm 1 Algorithme ID3

```
procedure ID3(S, Attributes)
    Créer un nœud Racine
    if tous les exemples de S sont de la même classe C then
        return Racine étiqueté avec la classe C
    end if
    if Attributes est vide then
        return Racine étiqueté avec la classe majoritaire de S
    end if
    Sélection de l'attribut :
    A ← l'attribut qui maximise le Gain d'Information
    Étiqueter Racine par A
    for chaque valeur possible v de A do
        S_v ← {x ∈ S | x_A = v}
        if S_v est vide then
            Ajouter une feuille avec la classe majoritaire de S
        else
            Ajouter le sous-arbre ID3(S_v, Attributes \ {A})
        end if
    end for
    return Racine
end procedure
```

Problèmes rencontrés (notebook)

- ID3 ne gère pas nativement les variables numériques.
- Encodage de labels : transforme le catégoriel en numérique, mais ne crée pas de seuils cohérents.
- Usage de `.unique()` : une branche par valeur ⇒ explosion du nombre de branches (ex: `Age`, `Na_to_K`).
- Surapprentissage : arbres profonds qui mémorisent le train set.

ID3 sur l'exemple météo (Quinlan 1989)

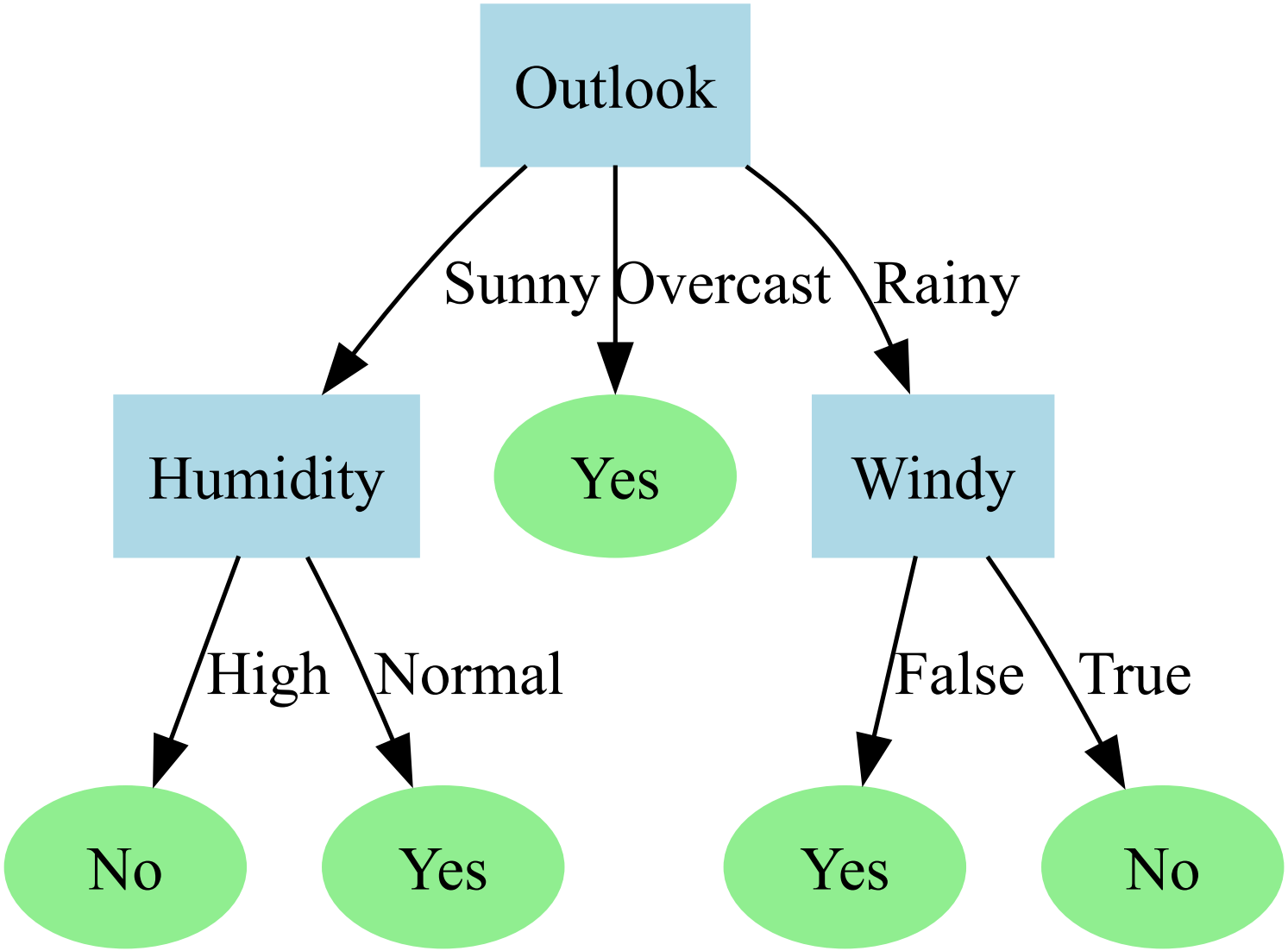

Figure 2.2.1: Arbre ID3 simplifié sur le dataset météo.

Dataset Drug200 : version complète vs réduite

- Version complète : `Age`, `Sex`, `BP`, `Cholesterol`, `Na_to_K`, `Drug`.
- Pour ID3, on a supprimé les variables numériques (`Age`, `Na_to_K`).
- But : éviter une branche par valeur et rendre l'arbre lisible.

Age	**Sex**	**BP**	**Chol**	**Na_to_K**	**Drug**
23	F	HIGH	HIGH	25.355	drugY
47	M	LOW	HIGH	13.093	drugC
47	M	LOW	HIGH	10.114	drugC
28	F	NORMAL	HIGH	7.798	drugX
61	F	LOW	HIGH	18.043	drugY
⋮	⋮	⋮	⋮	⋮	⋮
56	F	LOW	HIGH	11.567	drugC
16	M	LOW	HIGH	12.006	drugC
52	M	NORMAL	NORMAL	9.894	drugX
23	M	NORMAL	NORMAL	14.020	drugX
40	F	LOW	NORMAL	11.349	drugX

Figure 2.2.2: Extrait du dataset Drug200.

ID3 sur Drug200 : problème des variables continues

- Avec `.unique()`, une valeur numérique ⇒ une branche.
- Résultat : arbre très large et peu interprétable.

Figure 2.2.3: Exemple d'explosion de branches avec des variables continues.

Zoom sur une zone de l'arbre

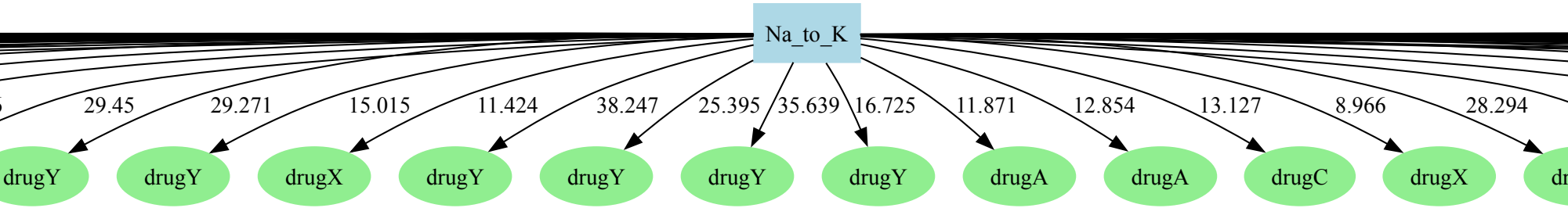

Figure 2.2.4: Zoom pour rendre la structure lisible.

ID3 sur Drug200 (variables catégorielles)

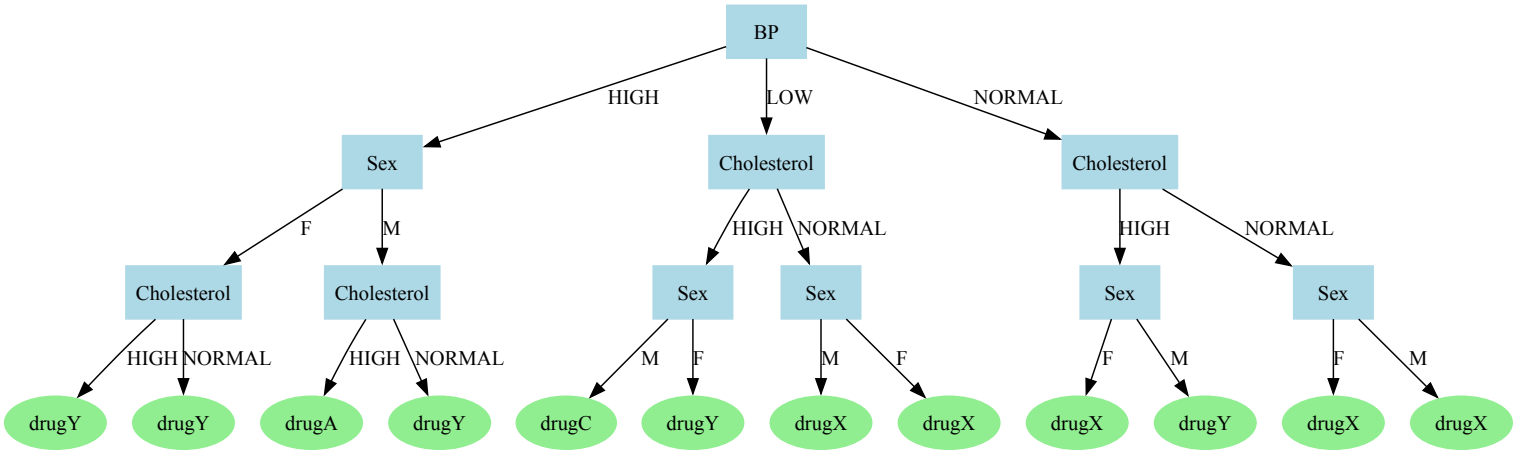

Figure 2.2.5: Exemple d'arbre ID3 sur Drug200 sans variables continues.

Pourquoi CART corrige ce problème

- CART gère les variables numériques en testant des seuils.
- Chaque split est binaire ($\leq t$ / $> t$), donc pas d'explosion de branches.
- Résultat : un arbre plus lisible et plus stable.

Sommaire

1. Histoire des arbres de décisions

2. Construction d'arbre
2.1 Principe de construction
2.2 Iterative Dichotomiser 3 (ID3)
2.3 Classification and Regression Trees (CART)

3. Élagage d'arbre

4. Random Forest

5. Limites et conclusion

Classification and Regression Trees (CART)

Algorithm 2 Algorithme CART

```
procedure CART(D)
    if critère d'arrêt atteint (pureté, profondeur max, etc.) then
        return nœud feuille avec la classe majoritaire
    end if
    Recherche de la séparation optimale :
    (j, t) ← Trouver l'attribut j et le seuil t minimisant l'indice de Gini
    Division binaire :
    D_gauche ← {x ∈ D | x_j ≤ t}
    D_droite ← {x ∈ D | x_j > t}
    Créer un nœud Racine divisé par le test (x_j ≤ t)
    Racine.gauche ← CART(D_gauche)
    Racine.droite ← CART(D_droite)
    return Racine
end procedure
```

CART sur l'exemple météo (Quinlan 1989)

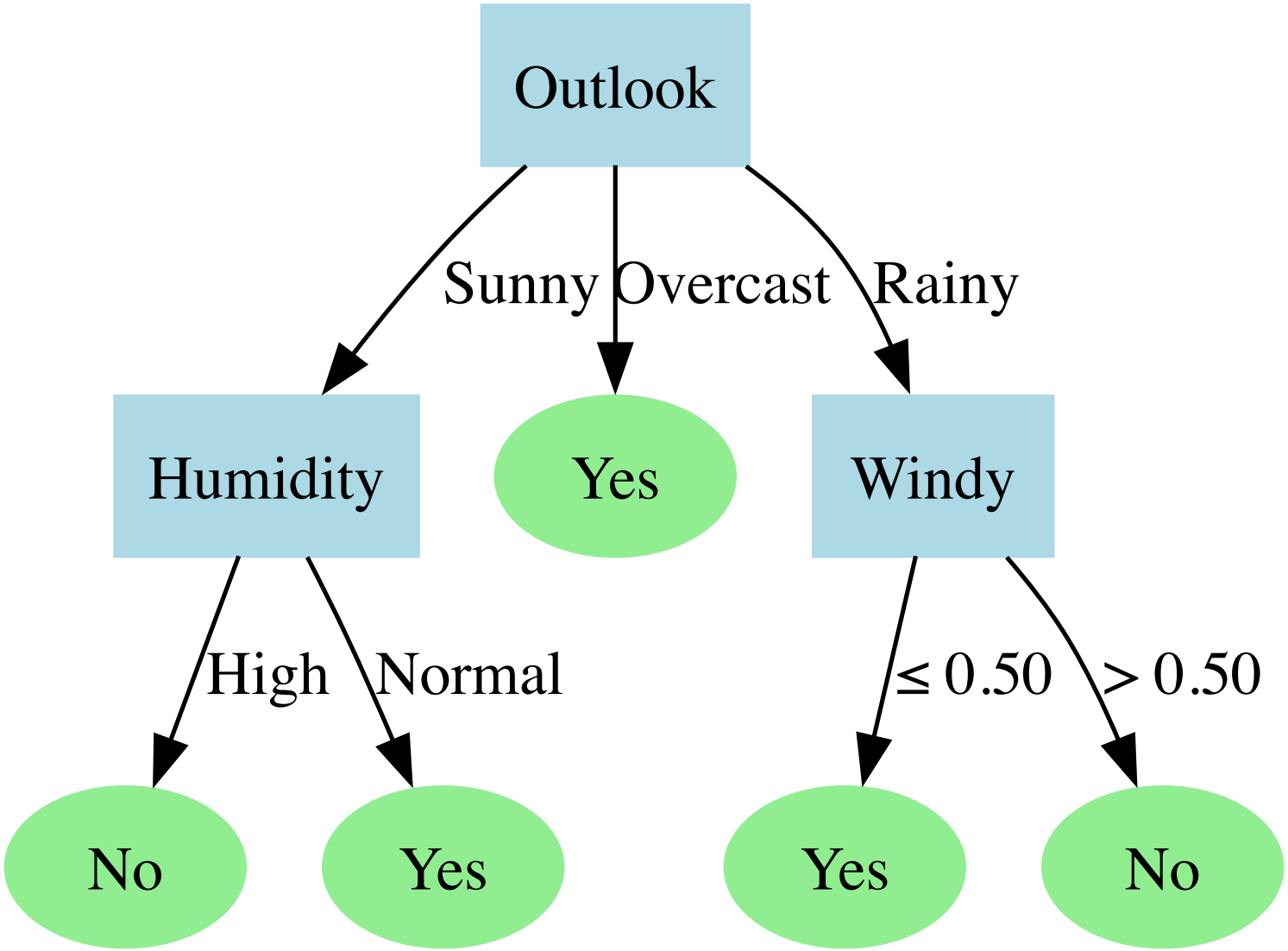

Figure 2.3.1: Exemple de CART sur le dataset météo.

CART sur Drug200

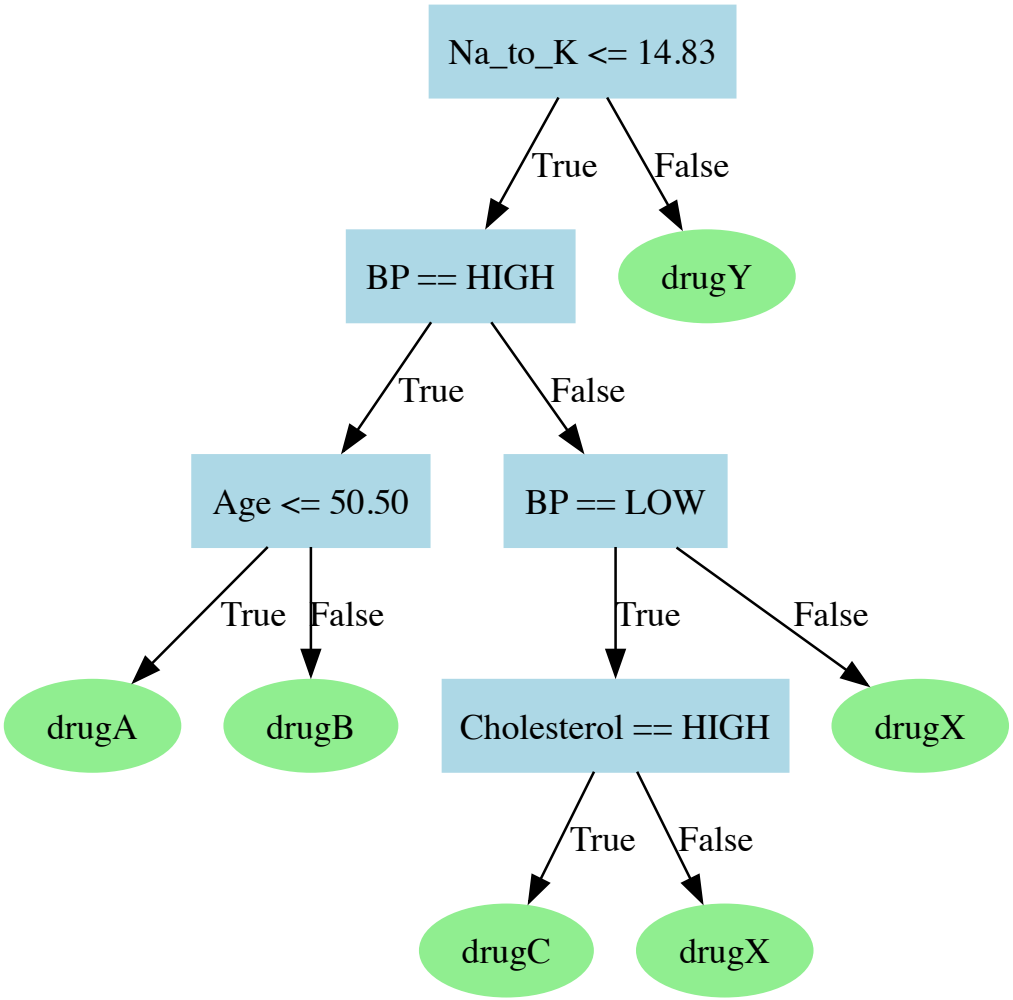

Figure 2.3.2: Exemple de CART sur le dataset Drug200.

Dataset Breast Cancer : aperçu

Age	Race	T	N	Grade	Tumor	Nodes+	Status
68	White	T1	N1	3	4	1	Alive
50	White	T2	N2	2	35	5	Alive
58	White	T3	N3	2	63	7	Alive
58	White	T1	N1	3	18	1	Alive
47	White	T2	N1	3	41	1	Alive
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
62	Other	T1	N1	2	9	1	Alive
56	White	T2	N2	2	46	8	Alive
68	White	T2	N1	2	22	3	Alive
58	Black	T2	N1	2	44	1	Alive
46	White	T2	N1	2	30	2	Alive

Figure 2.3.3: Extrait du dataset Breast Cancer.

CART sur Breast Cancer

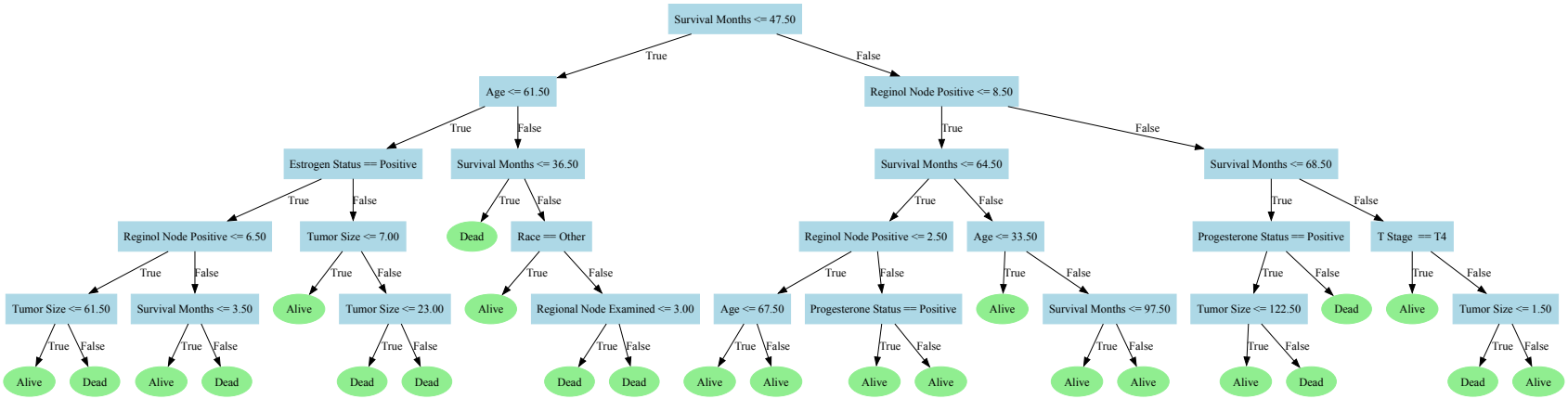

Figure 2.3.4: Exemple de CART sur un dataset médical.

ID3 vs CART : Synthèse

Caractéristique	**ID3**	**CART**
Critère	Entropie	Gini
Type d'arbre	Multi-branches	Binaire uniquement
Tâches	Classification	Classif. & Régression
Élagage	Non	Oui (Pruning)

Sommaire

1. Histoire des arbres de décisions
2. Construction d'arbre
3. Élagage d'arbre
4. Random Forest
5. Limites et conclusion

Pourquoi élaguer ?

- Un arbre trop profond mémorise les données d'entraînement.
- En test, il généralise moins bien : c'est le surapprentissage.
- L'élagage enlève des branches peu utiles pour rendre l'arbre plus simple.

Post-pruning (CART)

- Étape 1 : entraîner un arbre complet (très détaillé).
- Étape 2 : couper une branche candidate.
- Étape 3 : tester la précision sur un jeu de validation.
- Étape 4 : si la précision ne baisse pas, on garde la version simplifiée.
- On répète : moins de branches inutiles $\Rightarrow$ meilleure généralisation.

Pseudocode : élagage (post-pruning)

Notations : T = arbre courant, D_{val} = données de validation, T' = arbre simplifié.

Algorithm 3 Post-pruning (version simple)

```
procedure PRUNE(T, D_val)
    for chaque nœud interne n (du bas vers le haut) do
        T' ← arbre où n devient une feuille
        if accuracy(T', D_val) ≥ accuracy(T, D_val) then
            T ← T'
        end if
    end for
    return T
end procedure
```

Résultats élagage (placeholder)

Placeholder : précision train/val avant/après pruning

Figure 3.0.1: Impact de l'élagage sur la performance.

Sommaire

1. Histoire des arbres de décisions
2. Construction d'arbre
3. Élagage d'arbre
4. Random Forest
5. Limites et conclusion

Idée générale

- On entraîne plusieurs arbres différents.
- Chaque arbre voit un échantillon aléatoire des données (bootstrap).
- Chaque nœud teste un petit sous-ensemble de features.
- On combine les prédictions : vote majoritaire ou moyenne.

Schéma d'ensemble

Placeholder : schéma Random Forest

Paramètres usuels

- Nombre d'arbres n_trees.
- Profondeur max / taille minimale des feuilles.
- Sous-ensemble de features : $p = \sqrt{d}$ en classification.

Pseudocode : Random Forest

Algorithm 4 Random Forest (classification)

```
procedure RF(D, n_trees)
    for i = 1 à n_trees do
        D_i ← bootstrap(D)
        T_i ← CART(D_i) avec sous-ensemble aléatoire de features à chaque nœud
    end for
    return {T_1, ..., T_n_trees}
end procedure

procedure PREDICT({T_i}, x)
    prédictions ← {T_i(x)}
    return vote_majoritaire(prédictions)
end procedure
```

Résultats Random Forest (placeholder)

Placeholder : accuracy/erreur RF vs arbre seul

Figure 4.0.1: Comparaison Random Forest vs arbre unique.

Sommaire

1. Histoire des arbres de décisions
2. Construction d'arbre
3. Élagage d'arbre
4. Random Forest
5. Limites et conclusion

Limites

- Sensible au bruit et aux petites variations des données.
- Biais vers des attributs à nombreuses valeurs (ID3).
- Solutions : élagage, critères pénalisés, ensembles (Random Forest).

Conclusion

- Les arbres : interprétables et efficaces.
- ID3 : simple, entropie, multi-branches.
- CART : binaire, Gini, gère les variables continues + pruning.
- Perspectives : ensembles pour améliorer la robustesse.